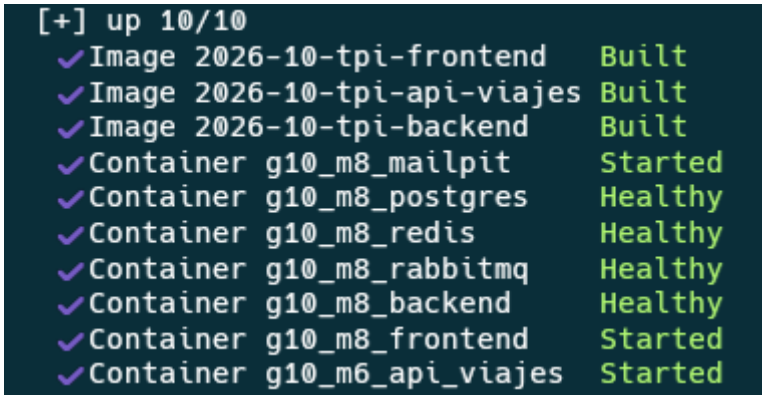
Levantar con un comando

docker compose up -d –build

(

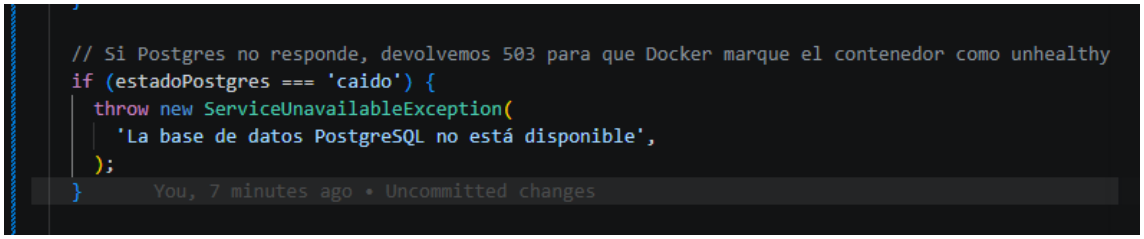
- -d detach, me deja usar la consola para escribir otros comandos,
- -- build reconstruye las imágenes, es decir que por así decirlo actualiza el código para evitar versiones viejas

)




HealthChecks y Reiniciar una dependencia

Dentro de apps/backend/src/modules/health/health.controller.ts




docker compose ps




Prueba con postgres caido

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (1.353s)
docker compose stop postgres

[+] stop 1/1
 ✔Container g10_m8_postgres Stopped


v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (2.652s)
curl.exe -i http://localhost:3000/api/v1/health

HTTP/1.1 503 Service Unavailable
```


```
{"type":"about:blank","title":"Servicio no disponible","status":503,"detail":"La base de datos PostgreSQL no está disponible","instance":"/api/v1/health","statusCode":503,"message":"La base de datos PostgreSQL no está disponible","error":"Service Unavailable","timestamp":"2026-11-10T04:09:58.605Z","path":"/api/v1/health"}
```


```
docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE      CREATED         STATUS                    PORTS
g10_m6_api_viajes   2026-10-tpi-api-viajes         "docker-entrypoint.s…"   api-viajes   5 minutes ago   Up 5 minutes (healthy)    0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
g10_m8_backend      2026-10-tpi-backend            "docker-entrypoint.s…"   backend      5 minutes ago   Up 5 minutes (unhealthy)  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
g10_m8_frontend     2026-10-tpi-frontend           "docker-entrypoint.s…"   frontend     5 minutes ago   Up 5 minutes              0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp
g10_m8_mailpit      axllent/mailpit:latest         "/mailpit"               mailpit      5 minutes ago   Up 5 minutes (healthy)    0.0.0.0:1025->1025/tcp, [::]:1025->1025/tcp,
025/tcp
g10_m8_rabbitmq     rabbitmq:3-management-alpine   "docker-entrypoint.s…"   rabbitmq     5 minutes ago   Up 5 minutes (healthy)    0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp,
->15672/tcp
g10_m8_redis        redis:7-alpine                 "docker-entrypoint.s…"   redis        5 minutes ago   Up 5 minutes (healthy)    0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
```

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.667s)
docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE      CREATED          STATUS
g10_m6_api_viajes   2026-10-tpi-api-viajes         "docker-entrypoint.s…"   api-viajes   6 minutes ago    Up 6 minutes (healthy)
g10_m8_backend      2026-10-tpi-backend            "docker-entrypoint.s…"   backend      6 minutes ago    Up 6 minutes (healthy)
g10_m8_frontend     2026-10-tpi-frontend           "docker-entrypoint.s…"   frontend     6 minutes ago    Up 6 minutes
g10_m8_mailpit      axllent/mailpit:latest         "/mailpit"               mailpit      6 minutes ago    Up 6 minutes (healthy)
25/tcp
g10_m8_postgres     postgres:16-alpine             "docker-entrypoint.s…"   postgres     6 minutes ago    Up 19 seconds (healthy)
g10_m8_rabbitmq     rabbitmq:3-management-alpine   "docker-entrypoint.s…"   rabbitmq     6 minutes ago    Up 6 minutes (healthy)
>15672/tcp
g10_m8_redis        redis:7-alpine                 "docker-entrypoint.s…"   redis        6 minutes ago    Up 6 minutes (healthy)
```

Prueba con Redis: No deberia decir unhealthy, sino que en lugar de usar Redis como cache, empieza a usar memoria local como cache

```
docker compose stop redis
[+] stop 1/1
 ✔Container g10_m8_redis Stopped

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.645s)
docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE      CREATED          STATUS
g10_m6_api_viajes   2026-10-tpi-api-viajes         "docker-entrypoint.s…"   api-viajes   8 minutes ago    Up 7 minutes (healthy)
g10_m8_backend      2026-10-tpi-backend            "docker-entrypoint.s…"   backend      8 minutes ago    Up 7 minutes (healthy)
g10_m8_frontend     2026-10-tpi-frontend           "docker-entrypoint.s…"   frontend     8 minutes ago    Up 7 minutes
g10_m8_mailpit      axllent/mailpit:latest         "/mailpit"               mailpit      8 minutes ago    Up 8 minutes (healthy)
->8025/tcp
g10_m8_postgres     postgres:16-alpine             "docker-entrypoint.s…"   postgres     8 minutes ago    Up About a minute (healthy)
g10_m8_rabbitmq     rabbitmq:3-management-alpine   "docker-entrypoint.s…"   rabbitmq     8 minutes ago    Up 8 minutes (healthy)
672->15672/tcp
```

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.991s)
docker compose stop redis

[+] stop 1/1
 ✔Container g10_m8_redis Stopped
                                                                    0.3s

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.056s)
curl.exe -i http://localhost:3000/api/v1/health

HTTP/1.1 200 OK
X-Powered-By: Express
X-Request-Id: 3ca42ffb-e5cf-40fe-8f94-53b5c60c7cbe
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 304
ETag: W/"130-ZOrwaUwsnSr+YstwvHTzRWl4c9M"
Date: Tue, 10 Nov 2026 04:17:04 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"estado":"ok","status":"degradado","module":"M8 - Notificaciones, Documentos y Soporte","modulo":"M8 - Notificaciones, Documentos y
 Soporte","group":"Grupo 10","dependencias":{"postgres":"activo","redis":"caido (usando memoria de respaldo)"},"timestamp":"2026-11-
10T04:17:04.140Z","uptime":624.85875168}
```

Reinicio de Redis (Si pasan menos de 10 segundos el backend se reconecta con redis, sino no lo vuelve a conectar y sigue con memoria local)

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (1.192s)
docker compose restart redis

[+] restart 0/1
 - Container g10_m8_redis Restarting

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.849s)
docker compose logs backend --tail 10

g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conectado a Redis en redis:6379
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [NestApplication] Nest application successfully started +1ms
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Servidor M8 iniciado en puerto 3000 (api/v1)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Documentación Swagger en http://localhost:3000/api/docs
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:38 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:21:35 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
```

Caso en que pasen más de 10 segundos

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.987s)
docker compose stop redis

[+] stop 1/1
 ✔Container g10_m8_redis Stopped

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.841s)
docker compose logs backend --tail 10

g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conectado a Redis en redis:6379
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [NestApplication] Nest application successfully started +1ms
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Servidor M8 iniciado en puerto 3000 (api/v1)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Documentación Swagger en http://localhost:3000/api/docs
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:38 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:21:35 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:22:32 AM    WARN [RedisService] Evento de error en Redis (getaddrinfo ENOTFOUND redis). Conmutando a memoria local.
```

A pesar de eso, el backend si hacemos el health check va a devolver código 200, eso fue una decisión de diseño

- **Si cae Postgres:** el backend no puede funcionar y responde **503**. Docker lo marca como unhealthy.
- **Si cae Redis:** el backend **sigue atendiendo** con una memoria local de respaldo y responde "degradado". El servicio sigue disponible.

Persistencia de datos:

En Redis

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.67s)
docker compose exec redis redis-cli SET prueba:persistencia "hola docker"
OK

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.93s)
docker compose stop redis
[+] stop 1/1
 ✔Container g10_m8_redis Stopped

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.865s)
docker compose start redis
[+] start 1/1
 ✔Container g10_m8_redis Started

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.57s)
docker compose exec redis redis-cli GET prueba:persistencia
"hola docker"
```

En postgres



```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (13s)
docker compose exec postgres psql -U postgres -d m8_db

psql (16.15)
Type "help" for help.

m8_db=# SELECT "receiptCode", "tripId", amount, "createdAt" FROM "DocumentReceipt" WHERE "tripId" ='123';
   receiptCode    | tripId | amount  |        createdAt
------------------+--------+---------+-------------------------
 REC-2026-XFT8569 | 123    | 3850.00 | 2026-11-10 04:38:47.178
(1 row)
```

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (9.598s)
docker compose down

[+] down 9/9
 ✔ Container g10_m6_api_viajes            Removed
 ✔ Container g10_m8_frontend              Removed
 ✔ Container g10_m8_backend               Removed
 ✔ Container g10_m8_mailpit               Removed
 ✔ Container g10_m8_postgres              Removed
 ✔ Container g10_m8_redis                 Removed
 ✔ Container g10_m8_rabbitmq              Removed
 ✔ Network 2026-10-tpi_m8_internal_net    Removed
 ✔ Network 2026-10-tpi_microservices_net  Removed
```

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (30.736s)
docker compose up -d

[+] up 9/9
 ✔ Network 2026-10-tpi_microservices_net  Created
 ✔ Network 2026-10-tpi_m8_internal_net    Created
 ✔ Container g10_m8_redis                 Healthy
 ✔ Container g10_m8_rabbitmq              Healthy
 ✔ Container g10_m8_mailpit               Started
 ✔ Container g10_m8_postgres              Healthy
 ✔ Container g10_m8_backend               Healthy
 ✔ Container g10_m6_api_viajes            Started
 ✔ Container g10_m8_frontend              Started
```



# COMPROBANTE DE VIAJE

Plataforma de Movilidad Urbana - Módulo 8

ID de viaje: 123
Fecha de emisión: 10/11/2026, 01:38:47
Pasajero: LeoMessi@gmail.com
Conductor: Roberto Gómez

**Total abonado: ARS 3850.00**

Comprobante digital de viaje.

Redes

En docker-compose

```yaml
networks:
  microservices_net: #Red donde van a hablar las APIs entre sí y con las dependencias
    driver: bridge
  m8_internal_net: #Red privada para el M8
    driver: bridge
```

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.195s)
docker network ls --filter name=2026-10-tpi

NETWORK ID     NAME                                DRIVER    SCOPE
1772d609fa91   2026-10-tpi_m8_internal_net          bridge    local
381c50dd2473   2026-10-tpi_microservices_net        bridge    local
```

Qué contenedor está en cada red

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.212s)
docker network inspect 2026-10-tpi_m8_internal_net --format "{{range .Containers}}{{.Name}} {{end}}"
g10_m8_backend g10_m8_postgres g10_m8_rabbitmq g10_m8_mailpit g10_m8_frontend g10_m8_redis

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.206s)
docker network inspect 2026-10-tpi_microservices_net --format "{{range .Containers}}{{.Name}} {{end}}"
g10_m8_backend g10_m8_rabbitmq g10_m6_api_viajes
```

Prueba de API Viajes al backend

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.687s)
docker compose exec api-viajes wget -qO- http://backend:3000/api/v1/health
{"estado":"ok","status":"ok","module":"M8 - Notificaciones, Documentos y Soporte","modulo":"M8 - Notificaciones, Documentos y Soporte","group":"Grupo 10","dependencias":{"postgres":"activo","redis":"activo"},"timestamp":"2026-11-10T04:47:20.791Z","uptime":321.457747646}
```

Prueba de API Viajes a la base de datos y a redis

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (2.875s)
docker compose exec api-viajes wget -qO- -T 3 http://postgres:5432
wget: bad address 'postgres:5432'
```

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (3.258s)
docker compose exec api-viajes wget -qO- -T 3 http://redis:6379
wget: bad address 'redis:6379'
```

Aclaracion: Por nombre, api-viajes no puede conectar con postgres ni redis porque no comparten red. Por IP sí llega, porque Postgres y Redis publican sus puertos en el host (5432, 6379) para poder usarlos en desarrollo, y Docker habilita el acceso a un puerto publicado desde cualquier origen. En producción no se publicarían esos puertos

Comunicación entre las 2 APIS

Finalizar Viaje:




Emitir Hito Canónico

```
{
  "origen": "M6 - API de Viajes (Contenedor 2)",
  "enviadoHacia": "http://backend:3000/api/v1/eventos",
  "xRequestId": "m6-req-1794286581056",
  "eventoEnviado": {
    "eventoId": "6e06f19c-092f-44fd-aede-100051d4742b",
    "tipo": "VIAJE_FINALIZADO",
    "origen": "M6",
    "ocurridoEn": "2026-11-10T04:56:21.056Z",
    "viajeId": "550e8400-e29b-41d4-a716-446655440100",
    "destinatarios": [
      {
        "usuarioId": "550e8400-e29b-41d4-a716-446655440999",
        "rol": "CLIENTE",
        "email": "alumno@utn.edu.ar",
        "canalesPreferidos": [
          "EMAIL"
        ]
      }
    ],
    "detalle": {
      "origen": "Campus UTN FRRe",
      "destino": "Plaza 25 de Mayo",
      "tipoVehiculo": "AUTO",
      "importe": {
        "monto": 5200,
        "moneda": "ARS"
      }
    }
  },
  "respuestaDeM8": {
    "eventoId": "6e06f19c-092f-44fd-aede-100051d4742b",
    "recibidoEn": "2026-11-10T04:56:21.061Z"
  }
}
```





La conexión entre las APIs en el código se ve en:

Docker compose

```yaml
api-viajes:
  build: ./apps/simulador-m6 # Construye la imagen con el Dockerfile de la carpeta del simulador
  container_name: g10_m6_api_viajes # Nombre del contenedor para identificarlo fácilmente
  restart: unless-stopped
  environment: # Sobreescribe solo estas del .env
    PUERTO: 3001
    URL_BACKEND_M8: http://backend:3000/api/v1
```

Dentro del simulador en

Apps/simulador-m6/servidor.js

```
3
4     const PUERTO = process.env.PUERTO || 3001;
5     // Apunta al servicio "backend" usando el DNS interno de la red de Docker
6     const URL_BACKEND_M8 = process.env.URL_BACKEND_M8 || 'http://backend:3000/api/v1';
```

```
75            // Llamada HTTP contenedor-a-contenedor hacia M8 (backend)
76            const respuesta = await fetch(`${URL_BACKEND_M8}/simulator/trip-completed`, {
77              method: 'POST',
78              headers: { 'Content-Type': 'application/json' },
79              body: JSON.stringify(datos),
80            });
```

Dentro del backend en

Apps/backend/src/modules/simulator/simulator.controller.ts

```
@Post('trip-completed')
@ApiOperation({ summary: 'Simular evento M6: Viaje Finalizado (TripCompletedEvent)' })
@ApiResponse({ status: 201, description: 'Evento simulado procesado en M8' })
async simulateTripCompleted(
  @Body()
  body: {
    tripId: string;
    passengerEmail: string;
    driverName: string;
    fare: number;
  },
) {
  const tripId = body.tripId || `trip-${Date.now()}`;
  const email = body.passengerEmail || 'pasajero.test@movilidad.com';
  const fare = body.fare || 4200.0;
  const driver = body.driverName || 'Martín Conductor';

  this.logger.log(`[SIMULADOR] Evento recibido: trip.completed para viaje ${tripId}`);
```